

04: The Just-In-Time Matching Engine

Abstract

The execution of continuous-time derivatives and programmatic order types (e.g., TWAP, limit orders) faces immense friction due to liquidity fragmentation and Maximal Extractable Value (MEV) leakage. This chapter introduces the Just-In-Time Matching (JTM) Engine, a substrate-agnostic execution protocol operating as a standalone Hub-and-Spoke architecture. By collapsing traditional $O(n)$ Limit Order Books into $O(1)$ pre-aggregated virtual "ghost" balances, the JTM Engine structurally internalizes the Coincidence of Wants (CoW), routing organic solver flow directly against dormant liquidity to strictly eliminate Loss-Versus-Rebalancing (LVR).

1. The TWAMM Microstructure Limitation

The foundational literature on Time-Weighted Average Market Makers (TWAMM) introduced by Paradigm (White, Adams, et al., 2021) established the standard for executing large institutional flow on-chain by breaking bulk orders into infinitely small virtual sub-orders. While this effectively mitigates front-running through between-block execution, the architectural design possesses a fundamental structural flaw: residual unmatched order flow is systematically executed against the embedded constant-product AMM.

1.1 Structural MEV Leakage

When a TWAMM executes unmatched flow against its underlying $x \cdot y = k$ curve, the passive maker is forced to absorb their own price impact. This design inherently relies on external arbitrageurs to correct the resulting price dislocation. Consequently, execution quality is fundamentally bounded by arbitrage efficiency, and the value of that correction leaks entirely to MEV extraction. To achieve optimal execution, the protocol must circumvent the passive AMM entirely.

2. The Interceptor: Just-In-Time Matching (JTM)

To eliminate this structural MEV leakage, the system introduces the "Interceptor" pattern via a central Hub. Rather than allowing organic taker flow to hit an underlying AMM directly, the `GhostRouter` (Hub) intercepts the flow and attempts to match it directly against the dormant "ghost" balances generated by continuous (`TwapEngine`) or discrete (`LimitEngine`) programmable Spoke engines.

Execution is strictly prioritized across a three-layer hierarchy:

1. L1 Internal Netting: The protocol aggregates opposing ghost streams (e.g., Maker A selling exactly while Maker B is buying) and cancels them at the oracle TWAP. This produces orthogonal cross execution with mathematically zero slippage.
2. L2 JIT Fill: Any directional remainder of ghost liquidity is directly intercepted by incoming market taker flow. The taker executes directly against the ghost pool.
3. L3 Clearing Auction: Any residual unexecuted ghost flow is exposed to a dynamic Dutch-auction discount to attract external solvers, replicating traditional TWAMM execution only as a mechanism of last resort.

Strict Pareto Improvement: The integration of the L2 JIT Fill transforms execution economics. Passive makers sell exactly at TWAP without incurring standard AMM trading fees, while active takers receive zero-slippage execution on the portion filled against the ghost pool.

3. Substrate-Agnostic Primitives

The core mathematical innovation of the JTM Engine is its substrate-agnostic treatment of liquidity. Whether liquidity originates from a streaming TWAP or a discrete limit order, upon activation, it is converted into a fungible, virtual "ghost" counter.

3.1 Continuous TWAP as JIT Limit Sequences

A TWAP stream accruing at a specific rate ($\text{sellRate} \times \Delta t$) produces a latent ghost balance waiting for execution. When a market taker arrives, this balance is matched at the current TWAP. Analytically, a TWAP stream operates identically to a continuous, infinitely dense sequence of instantaneous JIT limit orders where the protocol algorithmically dictates the price curve.

3.2 Verifiable Execution and the Ghost Limit Primitive

The structural fragility of pure intent-based off-chain execution protocols (e.g., CoW Protocol) was empirically demonstrated during the March 2026 \$50M aUSDT → aAAVE Trade Incident (Block 24643151) cascading failure ([AAVE Post Mortem](#), [CoWSwap Post Mortem](#)). When massive flow is forced through single-block "fill-or-kill" execution parameters, the system relies entirely on off-chain solvers successfully routing and landing transactions. As observed, compounding off-chain failures—stale gas ceilings rejecting optimal quotes, silent solver delivery failures, and public mempool leakage—result in catastrophic degradation where the protocol natively executes the *worst* available quote, inflicting severe value destruction on the user.

To mathematically prevent this execution toxicity, the JTM requires an explicitly verifiable, deterministic on-chain primitive: the Ghost Limit.

3.2.1 Deterministic Pre-Aggregation vs. Fill-Or-Kill Toxicity

Instead of attempting to force massive flow through an illiquid market instantly via external routing, a Ghost Limit pre-aggregates liquidity directly into the specialized `LimitEngine` Spoke. A depositor allocates tokens at a precise target tick. These tokens remain dormant until the 5-minute protocol TWAP physically crosses the target tick, acting as a strict anti-manipulation boundary against intra-block flash loan triggers.

Upon activation, the execution becomes mathematically deterministic. The tokens instantly join the aggregate ghost pool and become eligible for L2 JIT execution against *any* incoming flow. Because the execution logic is natively embedded within the Hub-and-Spoke architecture, it does not rely on a specific external solver winning an auction or successfully navigating gas limits. It executes incrementally and securely against organic volume exactly at the targeted tick, structurally capping maximum price impact.

3.2.2 Non-Reversible O(1) Fills

Unlike traditional on-chain limit orders (e.g., Uniswap V3 concentrated liquidity) that suffer from "bounce-back" vulnerabilities where a reversed price un-fills the order, Ghost Limit execution is unidirectional. Once matched, the earnings are mathematically distributed using a global accumulator algorithm. This allows for

$O(1)$ distribution complexity per claim, granting near-continuous price granularity (at `tickSpacing = 1`) without requiring the instantiation of actual LP positions.

3.3 The $O(1)$ Pre-Aggregation Collapse

By utilizing a global accumulator for earnings distribution, the JTM engine collapses a traditional $O(n)$ iterative Limit Order Book into a single virtual state variable.

When an external taker executes a market swap, the fill algorithm executes in absolute $O(1)$ constant time against the aggregate ghost balance. Because individual orders are pre-aggregated at the time of deposit rather than at the time of execution, external solvers are not required to compute complex routing trees.

- **The Trade-Off:** To achieve this computational efficiency, the protocol substitutes First-In-First-Out (FIFO) queueing with Pro-Rata distribution. All depositors resting at an activated tick share the aggregate fills proportionally. This guarantees fair execution for passive on-chain participants who cannot compete on latency.

4. Wallet-Permissioned Liquidity & Solver Routing

The Ghost Router serves as the ultimate on-chain execution layer for modern "Wallet-Permissioned Liquidity" frameworks ([Greenfield Capital, 2026](#)). In this paradigm, protocols access capital strictly at the time of execution via conditional approvals, eliminating pooled custody risk. The Ghost Router intrinsically operates on virtual balances, making it the ideal substrate for this model.

4.1 The No-Arbitrage Solver Routing Assumption

The JTM implicitly outsources the NP-hard optimization of Coincidence of Wants (CoW) to the external free market.

We define the **No-Arbitrage Assumption**: If the price on an external venue (e.g., Binance, CoWSwap, UniswapX) diverges from the internal ghost TWAP ($P_{External} \neq P_{Ghost}$), external solvers are economically incentivized to route order flow through the Ghost Pool to capture the spread.

Crucially, this solver-routed flow is mathematically treated by the system as organic L2 demand.

1. Limit depositors are filled precisely because the solver's swap triggers the JIT match.
2. Solvers capture the deterministic spread between the ghost TWAP and the external market price.
3. Market swappers on external venues receive tighter pricing as the arbitrageur suppresses the dislocation.

Consequently, the JTM operates as a simplified, on-chain CoWSwap meta-aggregator. By pre-aggregating conditional execution criteria into $O(1)$ virtual counters at the time of deposit, the system entirely removes solving complexity at the time of execution. The architecture is mathematically future-proof, capable of supporting limit, DCA, option, or intent-based execution within a singular, shared virtual environment.